

Wakacyjne Wyzwanie

Z  KN Solvro

Ostatni wykład!



- **Wstęp:** Podstawy GIT i GitHub - fundamenty współpracy.
- **REST API:** Czym jest API, kody stanu, metody HTTP.
- **Bazy danych:** Relacje, ORM, migracje - serce naszej aplikacji.
- **Manipulacja:** CRUD, kontrolery i dokumentacja (Swagger).
- **Dziś - Bezpieczeństwo:** Jak chronić to, co zbudowaliśmy. Auth, RBAC i zasady dostępu.



**Jak udowodnić,
że Ty to Ty?**



KN Solvro

Zasada Zero Zaufania

Dlaczego walidacja danych to absolutna podstawa bezpieczeństwa.



KN Solvro

Walidatory: Twoja pierwsza linia obrony



Nigdy nie ufaj klientowi

Niezależnie od tego, jak dobrą masz aplikację frontendową, wszystkie dane przychodzące na serwer z zewnątrz muszą być traktowane jako potencjalnie niebezpieczne.



Format i Typy

Sprawdzaj, czy email to na pewno email. Upewnij się, że wiek to liczba, a hasło ma odpowiednią siłę. Złe typy danych mogą wywołać krytyczne błędy (np. SQL Injection).



UX vs Bezpieczeństwo

Walidacja na frontendzie (HTML5, JS) służy wygodzie użytkownika (szybka informacja zwrotna).
Walidacja na backendzie gwarantuje bezpieczeństwo systemu.



KN Solvro



Dwa oblicza dostępu: AuthN vs AuthZ



Autentykacja (Authentication)

Odpowiada na pytanie: "Kim jesteś?"

To proces weryfikacji tożsamości użytkownika lub urządzenia. Udowadniasz serwerowi, że jesteś osobą, za którą się podajesz.

Przykłady: Login i hasło, odciski palców (biometria), kody SMS (2FA).



Autoryzacja (Authorization)

Odpowiada na pytanie: "Co możesz zrobić?"

Gdy system już wie kim jesteś, sprawdza, czy masz odpowiednie uprawnienia, aby wykonać daną akcję lub zobaczyć konkretne dane.

Przykłady: Zwykły użytkownik vs Administrator, role RBAC.



Nowoczesne Auth: JWT

JSON Web Token

W nowoczesnych API zrezygnowaliśmy z tradycyjnych, stanowych sesji (gdzie serwer musiał pamiętać każdego zalogowanego). Zamiast tego używamy bezstanowych tokenów.

Struktura JWT:

Header: Typ tokenu i algorytm szyfrowania.

Payload: Dane (np. ID użytkownika, role).

Signature: Podpis cyfrowy gwarantujący, że nikt nie zmienił zawartości!



KN Solvro

The screenshot shows the JWT.io web application in a browser. The URL is jwt.io. The interface includes a navigation bar with the JWT logo, a 'Debugger' button, and links to 'Libraries', 'Ask', and 'Get a T-shirt!'. A user profile 'Ryan' is in the top right. Below the navigation bar, there's a dropdown menu for 'ALGORITHM' set to 'HS256'. The main area is split into two columns: 'Encoded' and 'Decoded'. The 'Encoded' column contains a long string of base64-encoded characters. The 'Decoded' column shows the token's structure with 'HEADER' and 'PAYLOAD' sections. The 'HEADER' section contains a JSON object: `{ "alg": "HS256", "typ": "JWT" }`. The 'PAYLOAD' section contains a JSON object: `{ "sub": "1234567890", "name": "John Doe", "admin": true }`. Below these sections is a 'VERIFY SIGNATURE' section with a text input for a secret key and a checkbox for 'secret base64 encoded'. At the bottom, a blue button indicates 'Signature Verified'.

ALGORITHM: HS256

Encoded

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iNjY2OTY0MDE5InQJVA95OrM7E2cBab30RMHRhDcEfxjoYZgeFONFh7HgQ
```

Decoded

HEADER:

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD:

```
{  "sub": "1234567890",  "name": "John Doe",  "admin": true}
```

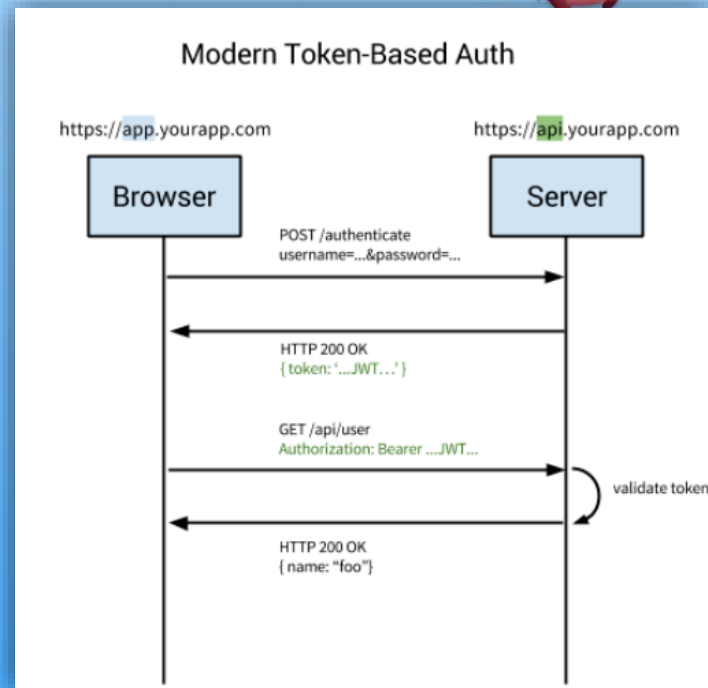
VERIFY SIGNATURE

HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),
secret
) ☐ secret base64 encoded

Signature Verified

Cykl życia żądania: Bearer Token

1. **Logowanie:** Klient wysłał POST /login. Serwer weryfikuje dane i odsyła podpisany JWT.
2. **Żądanie:** Klient dodaje nagłówek Authorization: Bearer <token> do każdego kolejnego zapytania HTTP.
3. **Przechwycenie:** Zapytanie trafia do API. Zanim dotrze do funkcji, zatrzymuje je w locie tzw. Guard (Strażnik).
4. **Weryfikacja:** Guard sprawdza podpis JWT i datę ważności. Jeśli token jest sfałszowany, odrzuca żądanie (Błąd 401).
5. **Wzbogacenie:** Jeśli token jest poprawny, Guard przypina odkodowane z niego dane (np. ID) bezpośrednio do obiektu Request.



RBAC

Role-Based Access Control - bo nie każdy może usunąć bazę danych.



KN Solvro

RBAC w praktyce



Rola Systemowa	Odczyt (GET)	Tworzenie (POST)	Edycja (PUT)	Usuwanie (DELETE)
Administrator	✓	✓	✓	✓
Moderator	✓	✗	✓	✓
Zalogowany Użytkownik	✓	✓ (własne)	✓ (własne)	✓ (własne)
Gość	✓	✗	✗	✗

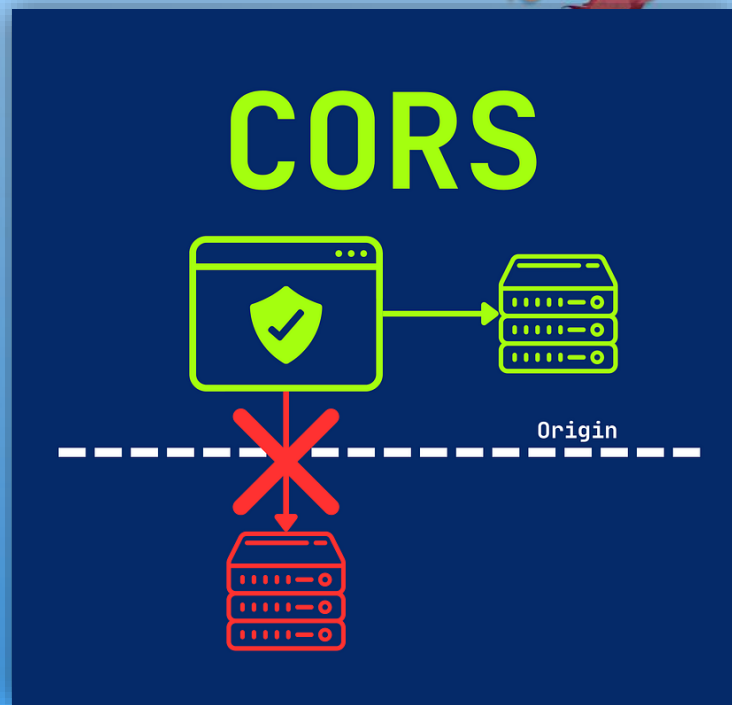


CORS: Zmora początkujących

Cross-Origin Resource Sharing

CORS nie jest "błędem" Twojego kodu. To wbudowany w przeglądarki internetowe mechanizm bezpieczeństwa.

- Domyślnie przeglądarka blokuje zapytania AJAX z innej domeny niż ta, na której działa strona.
- Chroni to użytkownika m.in. przed atakami typu CSRF (złośliwa strona nie może wysłać zapytania do banku w Twoim imieniu).
- Aby API pozwoliło na zapytania, musimy jawnie ustawić odpowiednie nagłówki (Access-Control-Allow-Origin).



KN Solvro

Złote zasady twórcy API



Zawsze HTTPS

Nigdy nie przysyłaj haseł, tokenów ani danych wrażliwych czystym tekstem. Szyfrowanie połączenia to absolutny obowiązek.



Waliduj wszystko

Przyjmij, że każdy request przysłany do Twojego kontrolera może być złośliwy. Sprawdzaj format, rozmiar i uprawnienia przed akcją.



Zasada min. przywileju

Użytkownicy i serwisy powinni mieć tylko tyle uprawnień (RBAC), ile jest im niezbędnie potrzebne do wykonania zadania. Nic więcej.



KN Solvro



KN Solvro

Magia NestJS: Jak to działa pod spodem?



@UseGuards()

Ochrona endpointu to zaledwie jeden dekorator nad kontrolerem. Guard automatycznie blokuje dostęp niezalogowanym i dba, by kod aplikacji wykonał się tylko dla zaufanych żądań.



Passport.js

NestJS wykorzystuje potężną bibliotekę Passport. To ona odwala "brudną robotę": wyciąga token z nagłówka, parsuje go, sprawdza algorytmy i automatycznie weryfikuje podpisy.



Obiekt @Req()

Dzięki Guardom, wewnątrz samego kontrolera mamy od razu gotowy obiekt req.user. Nie musimy sami dekodować tokenu – od razu wiemy, kto dokładnie wywołał akcję!





Dziękuję za uwagę

Cykl: Podstawy API zakończony. Czas na pytania!



KN Solvro